第 10 节 实操任务 3：添加审核循环

> **学习目标**：修改 review_node 使其返回 `review_passed: False`，观察 organize → review 循环最多执行 3 次 目标文件：`workflows/nodes.py`（修改）+ 验证 `workflows/graph.py`

背景

实操 2 搭建了完整工作流，但审核节点可能一次就通过了。本次实操我们通过**模拟审核失败**来验证条件边的循环行为。

关键问题：

- 审核未通过时，流程是否回到 organize？
- organize 是否读取了 review_feedback？
- iteration 是否正确递增？
- 第 3 次是否强制通过？

步骤 1：用 AI 编程工具修改 review_node

> 以下代码可以用 **OpenCode**、**Claude Code**、**Cursor**、**Trae** 或**通义灵码**等任意 AI 编程工具生成。

提示词：

```
请帮我修改 workflows/nodes.py 中的 review_node 函数，用于测试审核循环：

需求：
1. 前 2 次审核强制返回 review_passed: False（模拟审核不通过）
2. 第 3 次审核（iteration >= 2）返回 review_passed: True
3. 每次审核都给出不同的 feedback 内容
4. 打印当前 iteration 和 review_passed 值

这是临时测试代码，验证后会改回 LLM 审核版本。
```

生成的代码（参考实现）：

```
def review_node_test(state: KBState) -> dict:
    """审核节点（测试版）— 前 2 次强制失败，验证循环行为"""
    iteration = state.get("iteration", 0)

    # 模拟不同轮次的审核反馈
    feedbacks = [
```

```
        "摘要过于简短，需要补充技术细节和实际应用场景。",
        "标签不够精确，建议增加具体框架名称作为标签。",
        "质量已达标，准予通过。",
    ]

    if iteration >= 2:
        # 第 3 次强制通过
        passed = True
        feedback = feedbacks[2]
    else:
        # 前 2 次强制失败
        passed = False
        feedback = feedbacks[min(iteration, len(feedbacks) - 1)]

    print(f"[Reviewer-Test] 迭代 {iteration + 1}/3, passed={passed}")
    print(f"  反馈: {feedback}")

    return {
        "review_passed": passed,
        "review_feedback": feedback,
        "iteration": iteration + 1,
    }
```

步骤 2：临时替换 review_node

在 `workflows/graph.py` 中，临时将 review_node 替换为测试版：

```
# 临时测试：把 review_node 改为 review_node_test
from workflows.nodes import (
    analyze_node, collect_node, organize_node,
    review_node_test,  # 改这里
    save_node,
)

# graph.add_node("review", review_node)   # 注释掉
graph.add_node("review", review_node_test)  # 用测试版
```

步骤 3：理解代码

如果你对这段代码有疑问，可以让 AI 编程工具解释：

> `请解释审核循环的完整数据流：`
> `1. review_node 返回 review_passed=False 后，should_continue 怎么路由？`
> `2. organize_node 怎么读取 review_feedback 做修正？`
> `3. iteration 是在哪个节点递增的？`

> `4. 如果 review 一直返回 False，LangGraph 会不会死循环？`

关键理解：

问题	答案
should_continue 怎么路由？	读取 `state["review_passed"]`，False 时返回 `"organize"`
organize 怎么修正？	检查 `feedback` 非空且 `iteration > 0` 时调用 LLM 修改
iteration 哪里递增？	在 review_node 内，每次审核 +1
会不会死循环？	不会——iteration >= 2 时强制 passed=True

步骤 4：运行验证

```
cd ~/ai-knowledge-base
python workflows/graph.py
```

期望输出（关键部分）：

```
--- [organize] 完成 ---
[Reviewer-Test] 迭代 1/3, passed=False
  反馈: 摘要过于简短，需要补充技术细节和实际应用场景。

--- [review] 完成 ---
[Organizer] 整理出 6 条知识条目 (迭代 1)

--- [organize] 完成 ---
[Reviewer-Test] 迭代 2/3, passed=False
  反馈: 标签不够精确，建议增加具体框架名称作为标签。

--- [review] 完成 ---
[Organizer] 整理出 6 条知识条目 (迭代 2)

--- [organize] 完成 ---
[Reviewer-Test] 迭代 3/3, passed=True
  反馈: 质量已达标，准予通过。

--- [review] 完成 ---
[Saver] 保存 6 篇文章

--- [save] 完成 ---
```

验证清单：

检查项	期望	实际
organize 被调用 3 次	是（迭代 0, 1, 2）	
review 被调用 3 次	是（迭代 1/3, 2/3, 3/3）	
第 3 次 passed=True	是	
save 只执行 1 次	是（通过后）	
条件边正确路由	是	

步骤 5：恢复 LLM 审核版本

验证完成后，**一定要改回**正式的 review_node：

```
# 恢复正式版
from workflows.nodes import (
    analyze_node, collect_node, organize_node,
    review_node,      # 改回正式版
    save_node,
)

graph.add_node("review", review_node)  # 改回正式版
```

步骤 6：提交到 Git

```
git add workflows/nodes.py workflows/graph.py
git commit -m "test: verify review loop with 3 iterations, then restore
```

完成！ 条件边的审核循环验证通过。`review → organize → review` 最多循环 3 次，第 3 次强制通过。LangGraph 的 `add_conditional_edges` 正确处理了循环路由。